

Department of Computer Applications
II MCA – Concept Reinforcement Quiz 9

Aravindh Kannan M S
25mx304

1. Abstract Class Instantiation

Which statement is correct about the following code?

```
abstract class Animal {  
    abstract void sound();  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
    }  
}
```

- A. Program runs successfully
 - B. Compilation error because abstract classes cannot be instantiated
 - C. Runtime error
 - D. Abstract classes must contain constructors
-

2. Abstract Method Implementation

```
abstract class Shape {  
    abstract void draw();  
}
```

```
class Circle extends Shape { }
```

What happens when compiling?

- A. Runs successfully
 - B. Circle inherits draw() automatically
 - C. Compilation error because draw() is not implemented
 - D. Runtime exception
-

3. Abstract Method Rules

Which is **NOT allowed** for an abstract method?

- A. It must not have a body
 - B. It must be declared in an abstract class or interface
 - C. It can be declared final
 - D. It must be overridden in subclasses
-

4. Abstract Class with Constructor

```
abstract class Vehicle {  
    Vehicle() {  
        System.out.println("Vehicle created");  
    }  
}
```

Which statement is correct?

- A. Abstract classes cannot have constructors
- B. Constructors run when subclass object is created
- C. Compilation error
- D. Constructors must be abstract

5. Final Variable

```
class Test {  
    final int x = 10;  
  
    void change() {  
        x = 20;  
    }  
}
```

What happens?

- A. Compiles successfully
 - B. Runtime error
 - C. **Compilation error because final variable cannot change**
 - D. x becomes constant only after runtime
-

6. Final Method

```
class A {  
    final void display() {  
        System.out.println("A");  
    }  
}
```

```
class B extends A {  
    void display() {  
        System.out.println("B");  
    }  
}
```

Result?

- A. Works normally
 - B. Method overriding occurs
 - C. **Compilation error because final methods cannot be overridden**
 - D. Runtime exception
-

7. Final Class

```
final class A { }
```

```
class B extends A { }
```

Result?

- A. Compiles successfully
 - B. Runtime error
 - C. **Compilation error because final class cannot be inherited**
 - D. Class B overrides A
-

8. Interface Method Type

In Java interfaces, methods are by default:

- A. protected and abstract
 - B. **public and abstract**
 - C. private and abstract
 - D. public and static
-

9. Interface Variable

```
interface Test {  
    int x = 10;  
}
```

x is implicitly:

- A. **public static final**
 - B. private static
 - C. protected final
 - D. public volatile
-

10. Multiple Interface Implementation

```
interface A { void show(); }  
interface B { void show(); }
```

```
class Test implements A, B {  
    public void show() {  
        System.out.println("Hello");  
    }  
}
```

Result?

- A. Compilation error
 - B. **Works normally**
 - C. Runtime conflict
 - D. Must implement two show() methods
-

11. Abstract Class vs Interface

Which feature **only interfaces support** (traditional Java concept)?

- A. Constructors
 - B. **Multiple inheritance**
 - C. Instance variables
 - D. Protected methods
-

12. Abstract Class Method Body

```
abstract class A {  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

This code:

- A. Causes compilation error
 - B. **Is valid because abstract classes can have concrete methods**
 - C. Abstract methods must always exist
 - D. display() becomes abstract automatically
-

13. Interface Implementation

```
interface Shape {  
    void draw();  
}
```

class Circle implements Shape { }

Result?

- A. Compiles successfully
 - B. Compilation error because draw() must be implemented
 - C. Runtime exception
 - D. Interface methods are optional
-

14. Abstract Class Declaration

Which declaration is **valid**?

- A. abstract final class Test {}
 - B. abstract class Test {}
 - C. final abstract class Test {}
 - D. Both A and C
-

15. Final Reference Variable

```
class Test {  
    public static void main(String[] args) {  
        final StringBuilder sb = new StringBuilder("Hello");  
        sb.append(" World");  
        System.out.println(sb);  
    }  
}
```

Output?

- A. Compilation error
 - B. Hello
 - C. Hello World
 - D. Runtime error
-

16. Interface Instantiation

```
interface Test {  
    void show();  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Test t = new Test();  
    }  
}
```

Result?

- A. Works successfully
 - B. Runtime error
 - C. Compilation error because interface cannot be instantiated
 - D. show() runs automatically
-

17. Abstract Class Object Reference

```
abstract class A {  
    abstract void show();  
}
```

class B extends A {

```
void show() {  
    System.out.println("B");  
}  
}  
  
public class Test {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.show();  
    }  
}
```

Output?

- A. Compilation error
 - B. Runtime error
 - C. B**
 - D. A
-

18. Interface Inheritance

```
interface A {  
    void show();  
}
```

```
interface B extends A {  
    void display();  
}
```

Which is correct?

- A. Interface cannot extend another interface
 - B. Interface B inherits show()**
 - C. Compilation error
 - D. B must implement methods
-

19. Final Variable Initialization

```
class Test {  
    final int x;  
  
    Test() {  
        x = 20;  
    }  
}
```

This code:

- A. Causes compilation error
 - B. Is valid because final variable can be initialized in constructor**
 - C. Runtime exception
 - D. x must be initialized at declaration
-

20. Interface Method Implementation Access

```
interface A {  
    void show();  
}
```

```
class B implements A {
```

```
void show() {  
    System.out.println("Hello");  
}
```

Result?

- A. Works normally
- B. Compilation error because method must be public
- C. Runtime error
- D. Interface method becomes private